

Project 1 Report: Real-Time Video Filters

Name: Shreyas Sai Raman

Overall Project Description:

This project is a real-time C++ video filtering application built with OpenCV. The program captures live webcam frames and allows the user to switch between multiple image-processing modes using keyboard controls. The required filters include OpenCV grayscale, a custom grayscale effect, sepia tone, 5x5 blur, Sobel X/Y, gradient magnitude, blur/quantization, face detection, depth estimation with Depth Anything V2, and several creative effects. I also added extensions for slow-motion video recording and a caption overlay. The project helped me practice direct pixel access, convolution-style filters, signed image outputs, real-time display, saving images, using face detection, and incorporating an ONNX depth model into an OpenCV workflow.

Required Tasks and Images:

Image 1: OpenCV Grayscale



Original Image

Grayscale Image

This image shows the original video frame and the grayscale result produced by OpenCV's `cv::cvtColor` using `COLOR_BGR2GRAY`.

OpenCV uses the standard luminance formula: $Y = 0.299 R + 0.587 G + 0.114 B$

The green channel has the largest weight because human vision is most sensitive to green intensity.

Image 2: Alternative Grayscale



Alternative Grayscaled Image

This custom grayscale effect uses $255 - \text{red}$ as the grayscale value and copies that value into all three color channels. It looks different from OpenCV grayscale because it is based only on an inverted red channel rather than a weighted luminance calculation.

Image 3: Sepia Tone



Sepia Tone Image

The sepia filter gives the frame an antique-camera look. For each pixel, I stored the original B, G, and R values before computing the new channels. This ensures that the new red value is not accidentally reused while computing green or blue.

Image 4: 5 x 5 Blur



Original Image



Blurred Image

The blur filter uses the required 5x5 integer Gaussian approximation. I implemented both a naive full 5x5 version and a faster separable version.

Timing from my captured frame:

blur5x5_1: 87.3745 ms

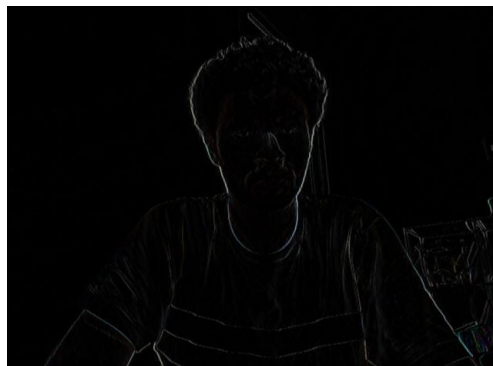
blur5x5_2: 14.966 ms

The second version is faster because it uses two 1x5 passes and direct row pointer access instead of repeatedly using `cv::Mat::at` inside a full 5x5 nested loop.

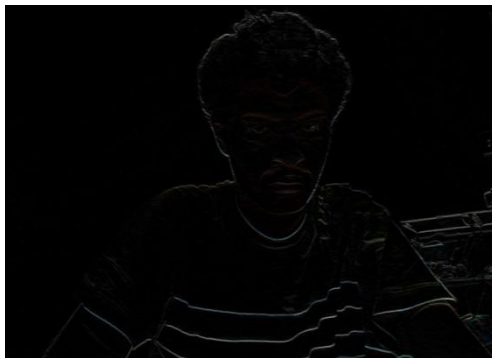
Image 5: Sobel X, Sobel Y and Gradient Magnitude



Original Image



Sobel X Image



Sobel Y Image



Gradient Magnitude Image

The Sobel X image highlights vertical edges, while Sobel Y highlights horizontal edges. The gradient magnitude image combines the X and Y signed outputs using Euclidean magnitude:

$$I = \sqrt{s_x^2 + s_y^2}$$

Image 6: Blur and Quantize



Original Image



Blur and Quantized Image

This effect first blurs the frame, then quantizes each color channel into a smaller number of levels. This reduces the number of possible colors and creates a posterized/cartoon-like appearance.

Image 7: Face Detection



Face Detected Image

This image shows the face detector running on the live video stream. Detected faces are outlined with a rectangle.

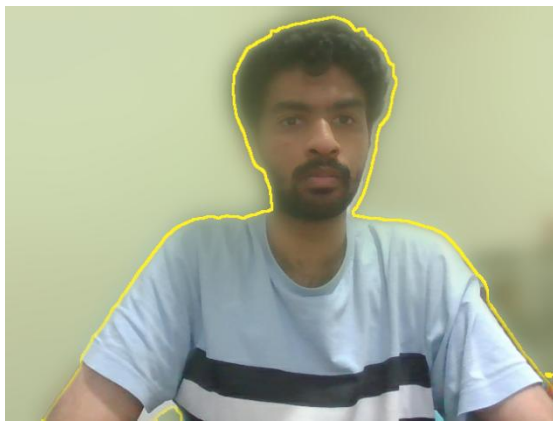
Image 8: Depth Image



Depth Image

This image shows the grayscale depth output from Depth Anything V2. Brighter and darker values represent relative depth estimates from the neural network, not metric distance.

Image 9: Creative Depth Filter



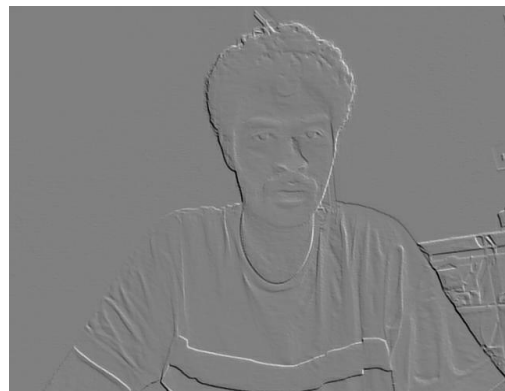
Creative Depth Filtered Image

This creative depth effect uses the depth map to keep nearer areas sharper while blurring and tinting farther areas. It also uses depth edges to add bright outlines.

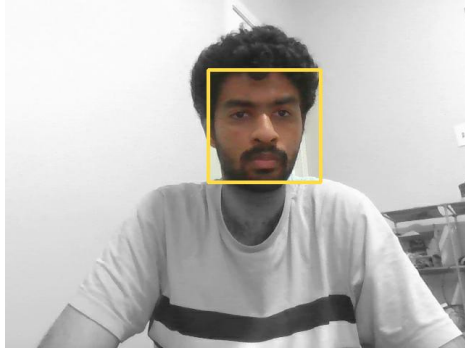
Image 10-12: Additional Effects



Color Negative



Embossing Effect



Colorful Face with Grayscale Background

Effect 10 is a color negative. It is a single-step pixel-wise transformation where each channel is replaced by $255 - \text{value}$.

Effect 11 is an emboss effect. It uses Sobel X and Sobel Y, then takes a directional dot product to create a raised gray relief look.

Effect 12 keeps the detected face in color while converting the rest of the frame to grayscale. This builds on the face detector.

Extensions:

Slow-Motion Video Recording

Google Drive Link:

https://drive.google.com/file/d/1zKEXeKE9vwXALs_e_Pg7_WlV5f4WVhq7/view?usp=drive_link

I added the ability to record short slow-motion video clips of the currently active effect. Pressing v starts recording, and pressing v again stops recording. The program creates slow motion by writing each processed frame twice. The recorded files are video-only.

Caption Overlay

Google Drive Link: https://drive.google.com/file/d/14kp-jjFjab513HV4wI7SE9Zh6NIPBQ_0/view?usp=drive_link

The application can be run with a caption:

```
.\vidDisplay.exe --caption "Depth is dramatic"
```

Pressing t toggles the caption overlay. When enabled, the caption appears near the bottom of the frame and is included in saved images and recorded videos. This works like a simple meme generator.

Reflection:

This project helped me understand how images are represented in OpenCV and how important image type and channel order are. I practiced accessing pixels directly, using signed intermediate images for Sobel filters, and separating processing images from display images. I also learned that faster implementations often come from reducing repeated work, such as using separable filters and pointer access. The depth model was the most challenging part

because it required combining OpenCV with ONNX Runtime, but it showed how neural network outputs can be used as another image channel for creative effects.

Acknowledgements:

I used the course-provided starter code and the provided DA2Network.hpp / model_fp16.onnx example for Depth Anything V2. I also consulted OpenCV documentation for cv::Mat, cv::cvtColor, cv::VideoCapture, cv::VideoWriter, and image saving/display functions. I used ChatGPT/Codex as an LLM assistant to help debug build issues, organize the code, generate README/report drafts, and reason through implementation steps. All final code was reviewed and tested in my project workspace.